IC 253 Assignment 1 Report

Region Detection and Binary Image Processing

Daksh Rathi B25132

1 Introduction

This report describes the implementation of two image-processing tasks using basic data structures and traversal algorithms. Q1 handles grayscale region detection and classification, while Q2 handles binary object detection, sparse representation, and inversion. The main focus is on the approach, the data structures used, and the resulting complexity.

2 Q1: Grayscale Region Detection and Shape Analysis

2.1 Objective

Q1 identifies non-background grayscale regions, removes small noisy groups, classifies significant regions from their geometric properties, and displays both textual and visual output.

2.2 Approach

The image is stored in a one-dimensional array, and a boolean array `visited` tracks whether a pixel has already been processed. During initialization, every pixel with intensity 0 is marked visited, while every non-zero pixel is left unvisited. This immediately excludes background pixels from later processing.

When an unvisited pixel is found, the helper function `buildComponent(start_row, start_col)` starts region construction. A stack is used to repeatedly pop the current pixel, denoted by `curr_pixel`, and inspect its four direct neighbours. A neighbour is included only if:

1. it lies inside the image,
2. it has not been processed earlier,
3. its intensity differs from the seed intensity by less than the threshold $T = 10$.

The comparison is always made with the seed intensity, which keeps the similarity rule consistent for the full region. As pixels are accepted, the program updates `minrow`, `maxrow`, `mincol`, `maxcol`, and `area`. After the full image scan, regions with area at most $0.01MN$ are excluded as noise, so only components larger than 1% of the image are retained. This prevents very small noisy regions from being treated as valid components.

2.3 Algorithm Summary

The overall Q1 algorithm can be summarized in four stages.

1. Read the grayscale matrix and initialize the `visited` array so that background pixels are ignored from the start.
2. Scan the image row by row and start `buildComponent()` from every unvisited non-zero pixel.
3. For each detected region, compute area and boundary-box attributes, then discard the region if its size is too small.
4. Classify each retained region using fill ratio and box proportions, draw the boundary boxes, and print the visual output.

2.4 Data Structures

The main structures are a **Point** class for coordinates, an array-based **Stack** for traversal, a **Component** class for storing boundary-box attributes and area, a result **linked list** for storing accepted regions and their type labels, and the **visited** array for inclusion or exclusion during scanning. These structures help manage traversal, store region information, and keep the final results."

2.5 Region Classification

Once the boundary box of a region is known, the program computes the area ratio

$$\text{Area Ratio} = \frac{\text{Area of region}}{\text{Area of boundary box}}.$$

This ratio is used with the box proportions to assign the final label.

- A region is treated as rectangular when its area ratio is at least 0.90. This indicates that the box is almost completely occupied.
- A region is treated as circular when the boundary box is nearly square and the area ratio lies between 0.60 and 0.85. This range is used because circles occupy less area than rectangles inside a bounding box.

After classification, `buildBoundary()` draws a box of intensity 255 around each accepted region, and `printVisualImage()` converts the result into ASCII form for terminal display.

2.6 Time and Space Complexity Analysis

Let the image size be $M \times N$.

Time Complexity

The image is scanned once to find starting points for region expansion. Each pixel can enter the stack at most once because the `visited` array prevents repeated expansion. During region

growth, only the four direct neighbours of the current pixel are checked, and the updates of area and boundary-box limits take constant time.

Therefore, the total running time remains linear in the number of image pixels:

$$O(M \times N)$$

Space Complexity

The auxiliary storage used during execution consists of:

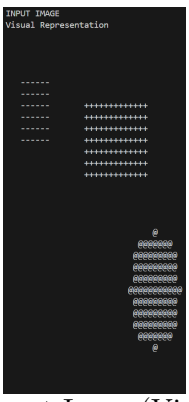
- the `visited` matrix of size $M \times N$,
- a stack used for region expansion, which in the worst case may contain $M \times N$ pixels,
- storage for the detected region summaries.

Hence, the total space complexity is:

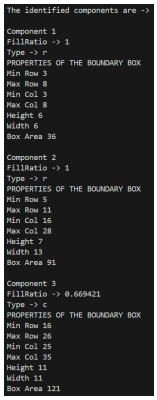
$$O(M \times N)$$

2.7 Test Cases

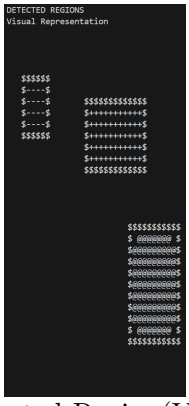
The Q1 program produces both textual and visual output. For report presentation, each selected test case may be shown using three images: the original visual input, the detected-region output, and the final visual form after marking the detected regions.



(a) Input Image(Visual)



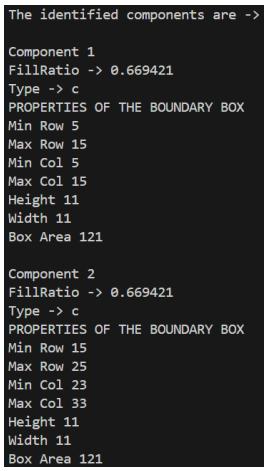
(b) Detected Region



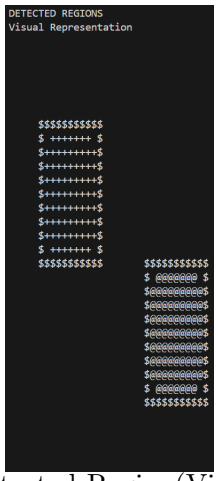
(c) Detected Region(Visual)

Figure 1: Test Case 1

(a) Input Image(Visual)



(b) Detected Region



(c) Detected Region(Visual)

Figure 2: Test Case 2

3 Q2: Binary Object Detection, Sparse Representation, and Inversion

3.1 Objective

Q2 detects foreground objects in a binary image, stores their boundary pixels, builds a sparse linked-list representation, inverts the image, and reconstructs the sparse form of the inverted result.

3.2 Approach

The binary image is stored in a dynamic array, and the `visited` array is initialized to `false` everywhere. A sparse linked list is then built by storing only those coordinates whose value is 1.

Detection begins from this sparse list. For each unvisited foreground coordinate, `identifyObject(start_row, start_col)` uses a stack to process the object one pixel at a time. The popped element `curr_pixel` checks its four direct neighbours. A valid unvisited neighbour with value 1 is included in the same object. A pixel is stored as a boundary pixel if at least one direct neighbour is outside the image or has value 0. During this traversal, the program also tracks the object area and boundary pixels. The inclusion rule is value 1, unvisited, and directly adjacent; the exclusion rule is background, out-of-bounds, or already processed.

More precisely, a foreground pixel is identified as a boundary pixel when inspection of its four direct neighbours finds even one neighbour that either lies outside the matrix limits or belongs to the background. If all four direct neighbours are valid foreground pixels, the current pixel is treated as an internal pixel rather than a boundary pixel.

For inversion, the full matrix is first set to 1, and then every coordinate present in the original sparse list is reset to 0. This yields the complement image directly.

3.3 Algorithm Summary

The Q2 algorithm proceeds in four major stages.

1. Read the binary image and create the sparse linked list containing only foreground coordinates.
2. Traverse the sparse list and start `identifyObject()` from each unvisited foreground coordinate.
3. For each object, compute its area and store all pixels that lie on the boundary.
4. Invert the image, rebuild the sparse representation for the inverted matrix, and print the updated output.

3.4 Data Structures

The main structures are **Point**, **Stack**, and **Object**, together with two linked lists: **Node_Sparse** for storing only foreground coordinates and **Node_Ans** for storing the detected objects. This reduces memory usage while still storing all foreground pixels. The sparse list is especially important because it avoids storing repeated information for background pixels.

3.5 Time and Space Complexity Analysis

Let the image size be $M \times N$.

Time Complexity

Constructing the sparse linked list requires one complete traversal of the binary image matrix. During object detection, each foreground pixel is visited at most once because it is marked immediately after being inserted into the stack, and each step checks only four direct neighbours. Flipping foreground and background values requires reconstruction of the updated image and one traversal of the sparse representation.

Therefore, the overall running time of the complete process is:

$$O(M \times N)$$

Space Complexity

The auxiliary storage used in this task includes:

- the `visited` matrix of size $M \times N$,
- a sparse linked list storing foreground-pixel information,
- a stack used for object expansion,
- temporary storage for object and point structures.

In the worst case, the number of foreground pixels may approach the total number of image cells, so the overall space complexity becomes:

$$O(M \times N)$$

3.6 Test Cases

The Q2 program prints multiple intermediate and final outputs, including the sparse representation, detected objects, and reconstructed image after inversion. For report presentation, it is clearer to include representative input-output figure pairs for selected test cases instead of printing the full terminal output everywhere.

```
10 12
0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0 0 0
0 0 1 1 1 0 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 1 1 1 0 0
0 0 0 0 0 0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0
```

(a) Input Binary Image

```
SHOWING THE ORIGINAL SPARSE LL
(1,3,1) (2,2,1) (2,3,1) (2,4,1) (3,3,1) (6,8,1) (7,7,1) (7,8,1) (7,9,1) (8,8,1)

FOLLOWING ARE THE IDENTIFIED OBJECTS

OBJECT 1
area(number of foreground pixels present in the identified object) -> 5
number of boundary pixels -> 4
The following are the boundary pixels ->
(1,3) (2,4) (2,2) (3,3)

OBJECT 2
area(number of foreground pixels present in the identified object) -> 5
number of boundary pixels -> 4
The following are the boundary pixels ->
(6,8) (7,9) (7,7) (8,8)


PRINTING THE FINAL RECONSTRUCTED IMAGE MATRIX
1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 0 1 1 1 1 1 1 1 1
1 1 0 0 0 1 1 1 1 1 1 1
1 1 1 0 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 0 1 1 1
1 1 1 1 1 1 1 0 0 0 1 1
1 1 1 1 1 1 1 1 0 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1

SHOWING THE NEW SPARSE LL(AFTER OPERATIONS)
(0,0,1) (0,1,1) (0,2,1) (0,3,1) (0,4,1) (0,5,1) (0,6,1) (0,7,1) (0,8,1) (0,9,1) (0,10,1) (0,11,1) (1,0,1) (1,1,1) (1,2,1) (1,4,1) (1,5,1) (1,6,1) (1,7,1) (1,8,1) (1,9,1) (1,10,1) (1,11,1
) (2,0,1) (2,1,1) (2,5,1) (2,6,1) (2,7,1) (2,8,1) (2,9,1) (2,10,1) (2,11,1) (3,0,1) (3,1,1) (3,2,1) (3,4,1) (3,5,1) (3,6,1) (3,7,1) (3,8,1) (3,9,1) (3,10,1) (3,11,1) (4,0,1) (4,1,1) (4,2
,1) (4,3,1) (4,4,1) (4,5,1) (4,6,1) (4,7,1) (4,8,1) (4,9,1) (4,10,1) (4,11,1) (5,0,1) (5,1,1) (5,2,1) (5,3,1) (5,4,1) (5,5,1) (5,6,1) (5,7,1) (5,8,1) (5,9,1) (5,10,1) (5,11,1) (6,0,1) (6
,1,1) (6,2,1) (6,3,1) (6,4,1) (6,5,1) (6,6,1) (6,7,1) (6,9,1) (6,10,1) (6,11,1) (7,0,1) (7,1,1) (7,2,1) (7,3,1) (7,4,1) (7,5,1) (7,6,1) (7,10,1) (7,11,1) (8,0,1) (8,1,1) (8,2,1) (8,3,1)
(8,4,1) (8,5,1) (8,6,1) (8,7,1) (8,9,1) (8,10,1) (8,11,1) (9,0,1) (9,1,1) (9,2,1) (9,3,1) (9,4,1) (9,5,1) (9,6,1) (9,7,1) (9,8,1) (9,9,1) (9,10,1) (9,11,1)
```

(b) Flipped Output Image

Figure 3: Test Case 1

```
10 15
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 1 1 1 1 0 0 0 0 0 0 0 0 0 0
0 1 1 1 1 0 0 0 0 0 0 0 0 0 0
0 1 1 1 1 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 1 1 1 1 0 0
0 0 0 0 0 0 0 0 0 1 1 1 1 0 0
0 0 0 0 0 0 0 0 0 1 1 1 1 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

(a) Input Binary Image

```
SHOWING THE ORIGINAL SPARSE LL
(1,1,1) (1,2,1) (1,3,1) (1,4,1) (2,1,1) (2,2,1) (2,3,1) (2,4,1) (3,1,1) (3,2,1) (3,3,1) (3,4,1) (5,9,1) (5,10,1) (5,11,1) (5,12,1) (6,9,1) (6,10,1) (6,11,1
) (6,12,1) (7,9,1) (7,10,1) (7,11,1) (7,12,1)

FOLLOWING ARE THE IDENTIFIED OBJECTS

OBJECT 1
area(number of foreground pixels present in the identified object) -> 12
number of boundary pixels -> 10
The following are the boundary pixels ->
(1,1) (1,2) (1,3) (1,4) (2,4) (3,4) (3,3) (3,2) (3,1) (2,1)

OBJECT 2
area(number of foreground pixels present in the identified object) -> 12
number of boundary pixels -> 10
The following are the boundary pixels ->
(5,9) (5,10) (5,11) (5,12) (6,12) (7,12) (7,11) (7,10) (7,9) (6,9)


PRINTING THE FINAL RECONSTRUCTED IMAGE MATRIX
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 0 0 0 0 1 1 1 1 1 1 1 1 1 1
1 0 0 0 0 1 1 1 1 1 1 1 1 1 1
1 0 0 0 0 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 0 0 0 0 1 1
1 1 1 1 1 1 1 1 1 0 0 0 0 1 1
1 1 1 1 1 1 1 1 1 0 0 0 0 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1

SHOWING THE NEW SPARSE LL(AFTER OPERATIONS)
(0,0,1) (0,1,1) (0,2,1) (0,3,1) (0,4,1) (0,5,1) (0,6,1) (0,7,1) (0,8,1) (0,9,1) (0,10,1) (0,11,1) (0,12,1) (0,13,1) (0,14,1) (1,0,1) (1,5,1) (1,6,1) (1,7,1
) (1,8,1) (1,9,1) (1,10,1) (1,11,1) (1,12,1) (1,13,1) (1,14,1) (2,0,1) (2,5,1) (2,6,1) (2,7,1) (2,8,1) (2,9,1) (2,10,1) (2,11,1) (2,12,1) (2,13,1) (2,14,1)
(3,0,1) (3,5,1) (3,6,1) (3,7,1) (3,8,1) (3,9,1) (3,10,1) (3,11,1) (3,12,1) (3,13,1) (3,14,1) (4,0,1) (4,1,1) (4,2,1) (4,3,1) (4,4,1) (4,5,1) (4,6,1) (4,7,
1) (4,8,1) (4,9,1) (4,10,1) (4,11,1) (4,12,1) (4,13,1) (4,14,1) (5,0,1) (5,1,1) (5,2,1) (5,3,1) (5,4,1) (5,5,1) (5,6,1) (5,7,1) (5,8,1) (5,13,1) (5,14,1) (
6,0,1) (6,1,1) (6,2,1) (6,3,1) (6,4,1) (6,5,1) (6,6,1) (6,7,1) (6,8,1) (6,13,1) (6,14,1) (7,0,1) (7,1,1) (7,2,1) (7,3,1) (7,4,1) (7,5,1) (7,6,1) (7,7,1) (7
,8,1) (7,13,1) (7,14,1) (8,0,1) (8,1,1) (8,2,1) (8,3,1) (8,4,1) (8,5,1) (8,6,1) (8,7,1) (8,8,1) (8,9,1) (8,10,1) (8,11,1) (8,12,1) (8,13,1) (8,14,1) (9,0,1
) (9,1,1) (9,2,1) (9,3,1) (9,4,1) (9,5,1) (9,6,1) (9,7,1) (9,8,1) (9,9,1) (9,10,1) (9,11,1) (9,12,1) (9,13,1) (9,14,1)
```

(b) Flipped Output Image

Figure 4: Test Case 2

4 GitHub Repository

The complete source code and associated test cases for both tasks can be accessed through the following repository links:

- Task 1: `https://github.com/dakshrathi-india/DSA-ASSIGNMENT-1/tree/main/Q1`
- Task 2: `https://github.com/dakshrathi-india/DSA-ASSIGNMENT-1/tree/main/Q2`